

生成 AI 利用申告書

- 氏名：塩澤 拓海
- 学籍番号：25G1065
- プログラムファイル名：HCK02_02.ino / HCK02_02.pde
- 使用した AI ツール：ChatGPT (GPT-5)、Claude Code (Opus 4.7)

1. 何に使ったか

提出課題 2 では、前回課題で作った楽曲を Processing で再生しつつ、マイクから入った波形を同じウィンドウに描画する必要がある。Arduino から Processing へどうデータを渡すか、Processing 側で受け取った値をどうやって時系列の波形として描けばよいか、の 2 点を生成 AI に相談した。

2. AI への指示（プロンプト）

Arduino でマイクの値を読んで、Processing のウィンドウに波形として描きたいです。Arduino 側はどう送って、Processing 側はどう受け取るのがシンプルですか？受け取った値を時間方向にスクロールさせて描く方法も教えてください。

3. AI の出力の使い方

- ☐ そのまま使った
- ☒ 一部修正して使った
- ☐ 参考にしただけ

→ 上で選んだ理由：

AI からは「Arduino 側は `Serial.println(d)` で 1 行 1 サンプル送る」「Processing 側は `bufferUntil('\n')` と `serialEvent` を組み合わせて、改行ごとに値を受け取る」「描画は配列に値を貯めておいて、毎フレーム左から右に並べて線で結ぶ」という説明があり、この方針はそのまま採用した。一方で、最初に提示されたサンプルではバイナリで 2 バイト送る案だったため、デバッグしづらいので文字列で送る形に変えた。楽曲再生の部分は前回課題の `HackInstrument` クラスをそのまま流用し、AI が新しく書いてきた音色生成コードは使わなかった。

4. 正しいと確認した方法

- Arduino IDE で `HCK02_02.ino` がコンパイルでき、シリアルモニタ (921600 bps) に数値が 1 行ずつ流れることを確認した
- Processing IDE で `HCK02_02.pde` を起動し、マイクに声を出すと白い線が縦に振れる様子を観察した
- キー p を押すと前回課題と同じ楽曲が PC のスピーカから流れ、その音をマイクに入れたときに波形がより大きく振れることを確認した
- キー 1~6 で音色を切り替えてから p を押すと異なる音色で再生されることを確認した
- Processing のコードを読み直し、循環バッファのインデックス計算 (`writeIdx + x`) % `samples.length` がなぜ「右端が最新」になるかを自分で説明できるようにした

5. 問題や間違い

- ☒ あった
- ☐ なかった

最初に AI が出してきたコードは、Processing 側で `port.read()` を `draw()` の中で呼ぶ書き方だった。これだと値の取りこぼしや描画ループとのタイミングずれが起きやすそうだったので、講義資料の例題 1 (`serialEvent` を使う書き方) に揃えた。また、起動直後に半端な文字列が流れてきたとき `Integer.parseInt` が例外を投げる場合があり、`NumberFormatException` を握りつぶす `try-catch` を入れて落ちないようにした。

6. 自分で工夫した点

- Arduino 側 (`HCK02_02.ino`) は、提出課題 1 のコードからプロッタ用の上下ガイドを取り除いて `Serial.println(d)` 1 本に絞り、Processing が一番シンプルに受け取れる形にした
- Processing 側 (`HCK02_02.pde`) は、前回課題の楽曲再生コード (`HackInstrument` クラス、メロディ・拍・音量の配列) をそのまま組み込み、波形描画と楽曲再生が同じ `p`, 1-6 のキー操作で扱えるようにした
- 波形は循環バッファを使い、配列の長さをウィンドウ幅 `width` と揃えることで、リサイズ時の対応や複雑な座標変換が要らない単純な構造にした

参考文献

- 有本泰子, 佐波孝彦「ハッカソン 1 第 2 回」講義資料 (`HCK1_02.pdf`, `HCK1_02_ex.pdf`)
- 生成 AI 利用申告書テンプレート (ハッカソン 1 講義資料)
- Arduino Reference: `Serial.println`, `analogRead`
<https://docs.arduino.cc/language-reference/>
- Processing Reference: `Serial`, `bufferUntil`, `serialEvent`
<https://processing.org/reference/libraries/serial/>
- Minim Reference: `AudioOutput`, `Oscil`, `Line`
<https://code.compartmental.net/minim/>